

“Cloud Based Online Voting System Using AWS”

PROJECT PHASE-II REPORT

Submitted by

**Mr. Varad Bhosale
Mr. Vaibhav Mungal
Mr. Mahesh Pujari
Mr. Nikhil Kokate**

*In partial fulfilment of the requirement for
the award of the degree of*

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING



**MAHATMA BASAVESHWAR EDUCATION SOCIETY'S COLLEGE
OF ENGINEERING, AMBAJOGAI.**

Affiliated To

**DR. BABASAHEB AMBEDKAR TECHNOLOGICAL UNIVERSITY,
LONERE, DIST. RAIGAD, MAHARASHTRA**

2025-2026

M. B. E. SOCIETY'S
COLLEGE OF ENGINEERING, AMBAJOGAI.
(Affiliated to Dr. Babasaheb Ambedkar Technological University,
Lonere)

CERTIFICATE

This is to Certify that the Project report titled “**Cloud Based Online Voting System Using AWS**” is the record of Bonafide work of “ **Mr. Varad Bhosale, Mr. Vaibhav Mungal, Mr. Mahesh Pujari, Mr. Nikhil Kokate** ” who carried out the Project Phase –II work under the supervision of **Prof P. V. Pawar** and submitted in fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING** affiliated to Dr. Babasaheb Technological University, Lonere, during academic 2025-26.

Prof. P. V. Pawar
Guide

Prof. Sushil V. Kulkarni
H.O.D

Dr. B. I. Khadakbhavi
Principal

ACKNOWLEDGEMENT

We would like to express our deep and sincere gratitude to our project Guide, **Prof. P. V. Pawar**, for giving us the opportunity to do project and providing invaluable guidance throughout this work. It was a great privilege and honor to work and study under his guidance.

We would like to express our deep and sincere gratitude to **Prof. Sushil V.Kulkarni**, HOD. For his motivation and kind support and for giving us the opportunity to carry out the Project Phase-II work.

We would like to express our deep and sincere gratitude to our Beloved Principal **Dr. B. I. Khadakbhavi, M.B.E.S College of Engineering, Ambajogai**, for his inspiration, motivation and his kind support to carry out our Project Phase-II work. Also, our special thanks to all our teachers for their kind support.

Finally, our thanks go to all the people who have supported us to complete the Project Phase-II work directly or indirectly.

Name:	PRN:
Mr. Varad Bhosale(88)	2221331242072
Mr. Vaibhav Mungal (82)	2221331242085
Mr. Mahesh Pujari (87)	2221331242123
Mr. Nikhil kokate (84)	2221331242127

ABSTRACT

The rapid advancement of digital technologies has created opportunities to modernize traditional systems, including voting processes that still depend heavily on physical infrastructure, manual counting, and human intervention. Traditional voting systems often suffer from challenges such as long queues, ballot mismanagement, human errors, limited accessibility, and delayed result processing. To overcome these limitations, the **Cloud-Based Online Voting System Using AWS** provides a secure, fast, scalable, and user-friendly digital platform designed to simplify the voting experience for both administrators and voters.

This project implements a complete web-based voting application using **HTML, CSS, JavaScript** for the frontend, **PHP** for backend processing, **MySQL** for structured data storage, and **Apache** as the web server environment. The system is hosted on **Amazon Web Services (AWS) EC2**, which ensures high availability, scalability, and robust performance even under large user loads. The proposed system allows voters to register, log in securely, view candidates, and cast their vote from any device with internet access. It strictly enforces the *one-user-one-vote* rule and prevents unauthorized access through secure authentication and server-side validation mechanisms.

Administrators are provided with a dedicated dashboard that enables them to manage voters, candidates, elections, and view real-time voting results. By automating core processes such as authentication, vote casting, result counting, and data storage, the platform significantly minimizes human effort and eliminates the possibility of manual errors or vote tampering.

The cloud-hosted infrastructure ensures that the system operates with minimal downtime and provides easy deployment, maintenance, and global accessibility. This makes the solution suitable for colleges, educational institutions, clubs, organizations, and small to medium-scale communities that require reliable and transparent election management.

Overall, this project demonstrates how cloud computing integrated with modern web technologies can deliver a secure, transparent, and efficient digital voting system. The system not only enhances user convenience but also contributes to reducing operational costs, improving accuracy, and accelerating decision-making processes.

TABLE OF CONTENT

CHAPTER NO	TITLE	PAGE NO
	ACKNOWLEDGEMENT	iii
	ABSTRACT	iv
	LIST OF FIGURES	vii
	SYSTEM DESIGN 4.1 SYSTEM ARCHITECTURE 4.2 AWS CLOUD ARCHITECTURE 4.2.1 AMAZON EKS & KUBERNETES ARCHITECTURE 4.3 SYSTEM MODULES 4.4 DATABASE DESIGN 4.5 VOTING WORKFLOW 4.6 ONE USER / ONE VOTING LIMITATION 4.7 SECURITY DESIGN & PORT GOVERNANCE 4.8 SYSTEM ADVANTAGES 4.9 SYSTEM LIMITATION 4.10 FUTURE ENHANCEMENT DESIGN	viii
1	IMPLEMENTATION 5.1 FRONT END IMPLEMENTATION 5.2 BACK END IMPLEMENTATION 5.3 DATABASE IMPLEMENTATION 5.4 DOCKER CONTAINERIZATION IMPLEMENTATION 5.5 KUBERNETES CLUSTER ORCHESTRATION 5.6 AUTOMATED JENKINS CI/CD & DEVSECOPS PIPELINE 5.7 INFRASTRUCTURE AS CODE (TERRAFORM) 5.8 TESTING IMPLEMENTATION 5.9 FINAL IMPLEMENTED SYSTEM	1-4
2.	TESTING & RESULT 6.1 TESTING OBJECTIVES 6.2 TYPES OF TESTING PERFORMED 6.3 UNIT TESTING 6.4 AWS CLOUD TESTING 6.5 CI/CD & DEVSECOPS AUTOMATED TESTING 6.6 TEST RESULT SUMMARY 6.7 SYSTEM OUTPUT 6.8 RESULT ANALYSIS	5-6
3.	SYSTEM ANALYSIS 3.1 HARDWARE REQUIREMENTS 3.2 SERVER-SIDE HARDWARE REQUIREMENTS 3.3 SOFTWARE REQUIREMENTS	7-13

	3.4 FUNCTIONAL REQUIREMENTS 3.5 NON-FUNCTIONAL REQUIREMENTS	
4.	SYSTEM DESIGN 4.1 SYSTEM ARCHITECTURE 4.2 AWS CLOUD ARCHITECTURE 4.3 SYSTEM MODULES 4.4 DATABASE DESIGN 4.5 VOTING WORKFLOW 4.6 ONE USER / ONE VOTING LIMITATION 4.7 SECURITY DESIGN 4.8 SYSTEM ADVANTAGES 4.9 SYSTEM LIMITATION 4.10 FUTURE ENHANCEMENT DESIGN	14-23
5.	IMPLEMENTATION 5.1 FRONT END IMPLEMENTATION 5.2 BACK END IMPLEMENTATION 5.3 DATABASE IMPLEMENTATION 5.4 AWS EC2 IMPLEMENTATION 5.6 TESTING 5.7 FINAL IMPLEMENTED SYSTEM	24-31
6.	TESTING & RESULT 6.1 TESTING OBJECTIVES 6.2 TYPES OF TESTING PERFORMED 6.3 UNIT TESTING 6.4 AWS CLOUD TESTING 6.5 TEST RESULT SUMMARY 6.6 SYSTEM OUTPUT 6.7 RESULT ANALYSIS	32-43
7.	CONCLUSION & FUTURE SCOPE 7.1 CONCLUSION 7.2 FUTURE SCOPE	44-45
	REFERENCES	46
	APPENDIX	47-49

LIST OF FIGURES

FIG.NO	TITLE	PAGE NO
3.1	USE CASE DIAGRAM	13
4.1	VOTING WORKFLOW	17
4.2	AWS CLOUD ARCHITECTURE	22
5.1	CLOUD BASED ONLINE VOTING SYSTEM ARCHITECTURE	25
5.2	ER DIAGRAM	30
6.1	USER LOGIN PORTAL	42
6.2	ADMIN LOGIN PORTAL	42
6.3	VOTER DASHBOARD SHOWING AVAILABLE ELECTIONS	43
6.4	ADMIN DASHBOARD	43

LIST OF TABLE

TABLE.NO	TITLE	PAGE NO
3.1	HARDWARE REQUIREMENTS	11
3.2	SOFTWARE REQUIREMENTS	12
3.3	NON-FUNCTIONAL REQUIREMENTS	12
6.1	TESTING OBJECTIVES	33
6.2	TYPES OF TESTING PERFORMED	34
6.3	UNIT TESTING	36
6.4	AWS CLOUD TESTING	37
6.5	TEST RESULT SUMMARY	39
6.6	SYSTEM OUTPUT	40
6.7	RESULT ANALYSIS	41

INTRODUCTION

1.1 Introduction

Voting plays an essential role in decision-making within democratic environments, educational institutions, organizations, and communities. Traditionally, voting processes have always required the physical presence of voters at designated locations, the use of paper ballots, and significant human labour for vote counting and validation. Although traditional voting methods have been widely accepted for many decades, they suffer from several limitations, including human errors, time consumption, and difficulty scaling to support larger populations.

As we move deeper into the digital age, there is an increasing need to adopt technological solutions that streamline these processes. One such solution is an **Online Voting System**—a secure digital platform that allows users to cast their votes electronically from any location. The introduction of digital voting significantly reduces human involvement, improves accuracy, and eliminates delays associated with manual vote processing.

To further enhance system reliability, this project leverages **Amazon Web Services (AWS)** through the EC2 cloud hosting platform. AWS provides scalable, secure, and highly available infrastructure capable of supporting modern web applications. Hosting the Online Voting System on AWS ensures that the platform is accessible from anywhere in the world with minimal downtime, making it ideal for institutions that require reliable online election processes.

This report provides a detailed explanation of the system, including its design, technical stack, architecture, features, limitations, and expected outcomes. The system is implemented using standard web technologies: **HTML**, **CSS**, **JavaScript** for frontend design, **PHP** for backend logic, **MySQL** as the database, and **Apache** as the server environment. Together, these technologies form a robust and scalable solution for online digital voting.

The purpose of this project is not only to digitize the election process but also to build trust through transparency, security, and automation. This report will detail every stage of developing.

1.2 PROBLEM STATEMENT

Traditional voting systems have several inherent weaknesses that make them inefficient for modern use. Despite being historically effective, they are no longer suitable for institutions handling frequent elections or large populations. Below are the major issues faced by conventional voting systems:

1. Physical Presence Requirement

Voters must visit polling stations to cast their votes, which restricts accessibility for individuals living far away, physically challenged individuals, and those who are unable to travel.

2. Long Queues and Time Consumption

The process of verifying voter identity, distributing ballots, casting votes, and counting results is labour-intensive and slow. It may take several hours—or even days—to finalize results.

3. Manual Errors and Data Manipulation

Human involvement in counting votes increases the risk of miscalculation. Furthermore, ballot tampering and duplicate voting create opportunities for manipulation.

4. Low Voter Turnout

Due to inconvenience or scheduling conflicts, many people may choose not to vote.

5. Costly Infrastructure

Arranging multiple polling booths, hiring staff, printing ballots, and securing physical locations contribute to high election costs.

6. Delayed Result Declaration

Manual tallying takes time, often delaying the announcement of results.

The goal of this project is to **solve these issues directly** by implementing a **cloud-based online voting platform** that is secure, scalable, cost-effective, and accessible from anywhere. This system eliminates the need for physical presence, speeds up the voting process, and provides results instantly.

1.3 OBJECTIVES

The Cloud-Based Online Voting System is developed with the following objectives:

Primary Objectives

To create a secure and user-friendly online voting platform.

To enable registered voters to cast their votes from any location using internet-enabled devices.

To automate result calculation by eliminating manual counting.

To ensure one-vote-per-user using strong authentication methods.

Technical Objectives

To build the system using powerful, widely used web technologies (PHP, MySQL, HTML, CSS, JavaScript).

To deploy the system on AWS EC2 for scalability and global accessibility.

To provide a responsive interface that works across desktops, laptops, and mobile devices.

To maintain a secure environment using encrypted storage, role-based access, and secure session management.

Outcome Objectives

To significantly reduce human errors.

To decrease the time taken to conduct elections and declare results.

To provide a transparent and reliable platform for institutions.

To demonstrate practical application of cloud computing in modern software systems.

1.4 SCOPE & LIMITATION

Scope

This Online Voting System is designed for academies, organizations, clubs, communities, and small to medium-scale elections. It can be used for:

Educational Institutions

College and university elections

Classroom representative elections

Departmental committee selections

Schools

Cultural decision voting

Class leader elections

Communities & Clubs

Event planning

Member selection

Leadership voting

Organizations

Employee polls

Internal decision-making processes

Project approval voting

General Benefits

Accessible from any internet-enabled device.

Highly scalable using AWS cloud technology.

Easily maintainable due to modular design.

Fast results with no manual intervention.

Limitations

Despite its broad applicability, this system has certain limitations:

Requires a stable internet connection.

Not designed for national elections without additional high-level security modules.

Dependent on registered user data accuracy.

2. LITERATURE SURVEY

This section reviews existing research papers and studies related to online voting systems, cloud computing, and security mechanisms. These works provide a strong foundation for designing a **secure, scalable, and reliable cloud-based voting system using AWS**.

2.1 Cloud-Based Secure Online Voting Framework (Gururaj et al., 2019)

A study proposed a **secure cloud-based framework for online voting systems**, focusing on encryption techniques such as AES, RSA, and Blowfish. The research highlighted that cloud environments improve **remote accessibility and scalability**, but require strong cryptographic mechanisms to ensure data security.

This work supports the use of **secure cloud infrastructure (AWS)** combined with encryption techniques in our system.

2.2 Online Voting System Using Homomorphic Encryption (Patil & Devmane, 2018)

This paper introduced an online voting system using the **Okamoto-Uchiyama homomorphic encryption algorithm**, allowing secure vote computation without revealing individual votes. The study emphasized confidentiality, accuracy, and scalability in cloud-based voting systems.

This research influenced our system's focus on **data confidentiality and secure vote processing**.

2.3 Cloud-Based Hybrid Blockchain Voting System (2024)

A recent study proposed a **hybrid cloud and blockchain-based voting system** to enhance transparency and security. It introduced phases such as voter registration, vote casting, and vote counting with **digital signatures and authentication protocols**.

This validates the need for **secure authentication and structured voting workflows** in our system.

2.4 Blockchain-Based Secure Voting System (Singh et al., 2024)

This research explored decentralized voting systems using blockchain technology to ensure **transparency, integrity, and elimination of intermediaries**. However, it also highlighted challenges such as complexity and scalability.

Our system adopts a **cloud-based centralized model using AWS**, which is simpler and more efficient for practical implementation.

2.5 Multi-Party Verifiable E-Voting Scheme (Wang et al., 2024)

This paper proposed a **secure and verifiable e-voting system** using blockchain and distributed storage (IPFS). It ensures voter anonymity through cryptographic techniques like ring signatures and separates vote counting from management processes.

This highlights the importance of **vote privacy and verifiability**, which are considered in our system design.

2.6 Distributed and Privacy-Preserving Internet Voting Systems (Chondros et al., 2019)

This study introduced distributed voting systems that ensure **end-to-end verifiability and voter privacy**. It demonstrated that distributed architectures improve scalability and allow voters to verify that their votes are correctly recorded.

This supports the use of **scalable cloud architectures like AWS EC2 and VPC**.

2.7 Secure Multiparty Computation in Cloud Voting (Chaudhari et al., 2018)

This research focused on using **secure multiparty computation (SMC)** in cloud-based voting systems to ensure that sensitive data is processed securely without exposing it.

This reinforces the importance of **secure data processing techniques** in cloud environments.

2.8 Distributed Internet Voting Architecture (Helm, 2021)

This study proposed a **thin-client distributed architecture** for online voting systems, covering the entire voting lifecycle including registration, voting, and result computation.

3. SYSTEM ANALYSIS

3.1 Hardware Requirements

The hardware requirements of the proposed system are divided into two categories: client-side requirements and server-side requirements. These resources ensure smooth operation of the voting platform and provide a reliable user experience.

Client-Side Hardware Requirements

The client-side hardware refers to the devices used by voters and administrators to access the system. Since the application is web-based, users do not need any specialized hardware or software installation. A standard internet-enabled device is sufficient to access the voting portal.

The minimum recommended hardware specifications include a Dual-Core processor, at least 2 GB RAM, and approximately 200 MB of available storage space for browser cache and temporary files. The device should support a display resolution of at least 1024×768 pixels to ensure proper viewing of the user interface. Input can be provided through a keyboard and mouse in desktop environments or through touchscreens on mobile devices. The system can be accessed using various devices such as smartphones, laptops, desktop computers, and tablets. A stable internet connection is required to ensure uninterrupted communication with the cloud-hosted application.

Server-Side Hardware Requirements

The server-side infrastructure is responsible for hosting the application, processing user requests, storing data, and managing voting operations. The proposed system is deployed on Amazon Web Services (AWS) using an EC2 instance.

A minimum AWS EC2 instance such as t2.micro or t2.small is sufficient for small to medium-scale elections. The server should provide 1–2 virtual CPUs, 1–2 GB RAM, and approximately 8–20 GB of SSD-based EBS storage. Ubuntu Linux or Amazon Linux can be used as the operating system. AWS Security Groups are configured to control network access and enhance security.

3.2 Server-Side Hardware Requirements (AWS Infrastructure)

The server-side infrastructure is responsible for hosting the application, processing user requests, storing election data, and managing the overall functionality of the Online Voting System. To ensure reliable performance and continuous availability, the proposed system is deployed on Amazon Web Services (AWS) using an EC2 instance.

The AWS EC2 instance serves as the primary hosting environment for the application. It runs the Apache web server, PHP backend, and MySQL database required for the execution of the voting platform. The cloud-based deployment enables users to access the system from any location with an internet connection while providing better scalability and availability compared to traditional on-premise hosting solutions.

For the successful operation of the system, a minimum AWS EC2 instance such as **t2.micro** or **t2.small** is sufficient. The server should provide **1–2 virtual CPUs (vCPUs)** and **1–2 GB of RAM**, which are adequate for handling user authentication, vote processing, and database operations during college-level elections. Storage requirements are fulfilled through **Amazon Elastic Block Store (EBS)**, which provides approximately **8–20 GB SSD storage** for the operating system, application files, and database records.

3.3 Software Requirements

The software requirements include all technologies, tools, frameworks, and cloud services used in the development and deployment of the Online Voting System.

Client-Side Software

Users access the system through standard web browsers such as Google Chrome, Mozilla Firefox, Microsoft Edge, or Safari. The frontend interface has been developed using HTML5, CSS3, and JavaScript. These technologies help create an interactive, responsive, and user-friendly environment that works across multiple platforms and devices.

Server-Side Software

The backend of the system has been developed using PHP, which handles business logic, authentication, vote processing, and database communication. Apache HTTP Server is used to host and execute PHP scripts. MySQL is used as the relational database management system for storing voter records, candidate information, election details, and voting data. phpMyAdmin is utilized for database administration and management.

Cloud Services

Several AWS services have been incorporated into the project. Amazon EC2 is used to host the application in the cloud environment. AWS IAM provides secure access management and permission control for cloud resources. AWS Security Groups act as a virtual firewall that controls inbound and outbound traffic. AWS S3 is used for storing candidate images and other uploaded files, while AWS EBS provides persistent storage for the EC2 instance.

Development Tools

The project was developed using Visual Studio Code as the primary code editor. Git and GitHub were used for version control and source code management. Browser Developer Tools were used for debugging and frontend testing throughout the development process.

3.4 Functional Requirements

Functional requirements define the features and operations that the system must perform in order to meet user expectations.

Admin Functional Requirements

The administrator is responsible for managing the overall election process. The system provides secure administrator authentication to ensure that only authorized personnel can access administrative features.

After successful login, the administrator can create new elections, activate or deactivate elections, and manage election details. The administrator can add new candidates, update candidate information, and upload candidate photographs.

Another important responsibility of the administrator is viewing election results. The system automatically counts votes and generates election results, reducing manual effort and minimizing the possibility of calculation errors.

Voter Functional Requirements

The voter module allows users to participate in elections through a secure and simple interface. New users can register by providing the required information. The system validates user details before creating an account.

Registered users can log in using their credentials and access the voting dashboard. Voters can view available elections and candidate information before casting their vote. The system enforces a one-user-one-vote policy, ensuring that each voter can vote only once in a particular election.

3.5 Non-Functional Requirements

Non-functional requirements describe how the system should perform rather than what it should do. These requirements focus on quality attributes such as performance, security, reliability, and usability.

Performance Requirements

The system should provide quick response times and support multiple users simultaneously without significant performance degradation. Web pages should load within a few seconds under normal operating conditions, and database queries should be executed efficiently to ensure smooth voting operations.

Security Requirements

Security is a critical requirement for any online voting application. The system implements secure authentication mechanisms, session management, and access control to protect user accounts and administrative functions. Duplicate voting attempts are prevented through vote validation mechanisms. Database access is controlled to protect sensitive information from unauthorized users.

Reliability Requirements

The system should remain available throughout the election period with minimal downtime. The use of AWS cloud infrastructure enhances reliability by providing stable hosting and continuous service availability.

Usability Requirements

The application should be easy to use for both administrators and voters. The interface has been designed with simplicity in mind so that users with basic computer knowledge can navigate the system without difficulty.

Scalability Requirements

The cloud-based architecture allows the system to be expanded in the future to support larger numbers of users and elections. Additional AWS resources can be allocated whenever increased performance or storage capacity is required.

TABLE 3.1 HARDWARE REQUIREMENTS

TYPE	COMPONENTS	SPECIFICATION
CLIENT-SIDE	PROCESSOR RAM STORAGE DISPLAY DEVICE INTERNET	DUAL CORE (INTEL/AMD/ARM) MINIMUM 2 GB 200 MB SPACE 1024×768 RES SMARTPHONE, DESKTOP STABLE CONNECTION
STABLE CONNECTION SERVER-SIDE (AWS EC2)	INSTANCE VCPU RAM STORAGE OS	T2.MICRO / T2.SMALL 1–2 1–2 GB 8–20 GB SSD UBUNTU / AMAZON LINUX

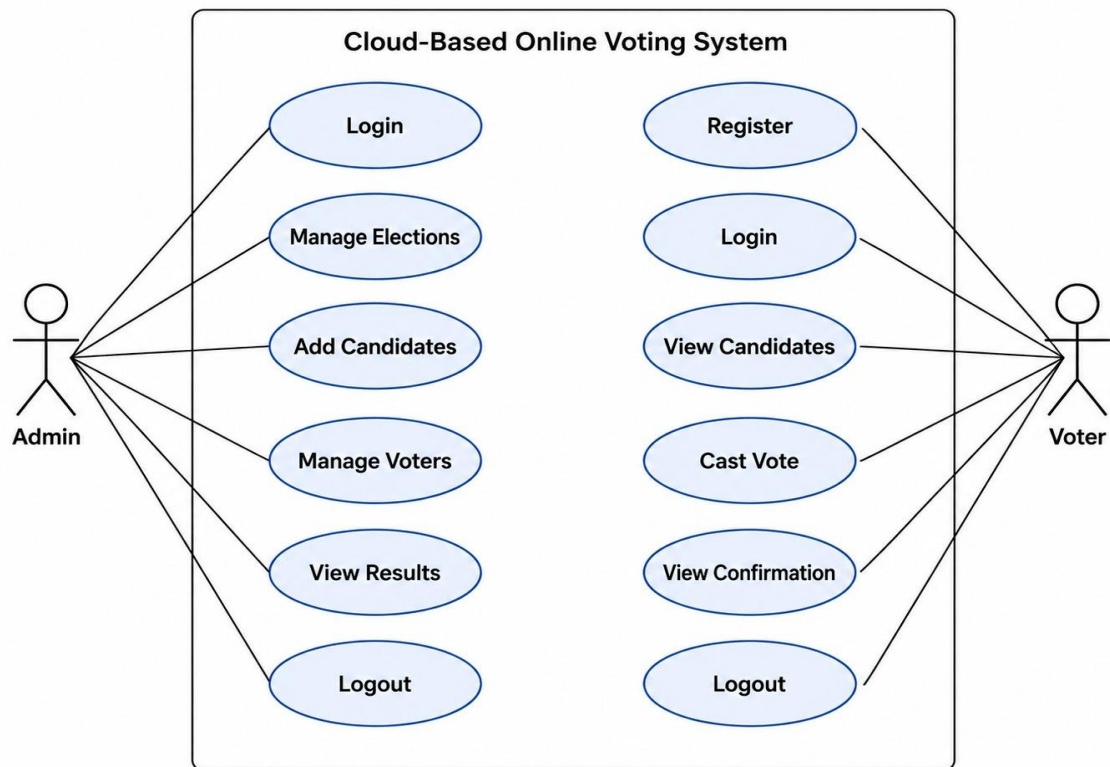
TABLE 3.2 SOFTWARE REQUIREMENTS

TYPE	SOFTWARE/TECHNOLOGY	PURPOSE
CLIENT-SIDE	CHROME, FIREFOX,EDGE,SAFARI HTML5, CSS3, JAVASCRIPT	WEB BROWSERS FRONTEND DEVELOPMENT
SERVER-SIDE	PHP 7/8 APACHE MYSQL PHPMYADMIN	BACKEND LOGIC WEB SERVER DATABASE DB MANAGEMENT
AWS SERVICE	EC2 IAM S3 SECURITY GROUPS	HOSTING ACCESS CONTROL FILE STORAGE FIREWALL
TOOLS	VS CODE GITHUB POSTMAN	DEVELOPMENT VERSION CONTROL API TESTING

TABLE 3.3 NON-FUNCTIONAL REQUIREMENT

CATEGORY	DESCRIPTION
PERFORMANCE	SUPPORTS MULTIPLE USERS, FAST RESPONSE (<3 SEC)
SECURITY	AUTHENTICATION, NO DUPLICATE VOTING, PROTECTION FROM SQL INJECTION
RELIABILITY	CONTINUOUS AVAILABILITY, STABLE AWS HOSTING
SCALABILITY	CAN HANDLE LARGE ELECTIONS (AWS SCALABLE)
USABILITY	SIMPLE, MOBILE-FRIENDLY INTERFACE
MAINTAINABILITY	MODULAR CODE, EASY UPDATES, GIT SUPPORT

Use Case Diagram for Online Voting System



3.1 Use Case Diagram

Use Case Diagram Description

The use case diagram represents the Cloud-Based Online Voting System with two main actors: Admin and Voter.

The Admin can manage elections, add candidates, manage voters, view results, and logout.

The Voter can register, login, view candidates, cast votes, view confirmation, and logout.

The system is developed using PHP, SQL database, and hosted on AWS cloud infrastructure for secure and efficient online voting.

4. SYSTEM DESIGN

The System Design chapter describes the internal architecture, workflow, modules, database structure, and cloud deployment of the Cloud-Based Online Voting System Using AWS. The system is designed using a modular web-based architecture to ensure security, scalability, maintainability, and ease of use.

The project uses PHP for backend processing, MySQL for database management, HTML/CSS/JavaScript for frontend development, and AWS EC2 cloud infrastructure for hosting and deployment.

4.1 SYSTEM ARCHITECTURE

The Online Voting System follows a three-tier architecture model that separates the user interface, business logic, and data storage components. This separation improves maintainability, security, and scalability of the application.

The first layer is the Presentation Layer, which acts as the user interface of the system. It is developed using HTML5, CSS3, JavaScript, and Bootstrap. This layer provides all the pages that users interact with, including registration, login, dashboard, candidate list, voting page, and result display pages. The interface is designed to be responsive so that it can work efficiently on desktops, laptops, tablets, and smartphones.

The second layer is the Application Layer, which contains the core business logic of the system. This layer is implemented using PHP and Apache Web Server. It handles user authentication, session management, vote validation, election management, candidate management, and result generation. Whenever a user performs an action, the request is processed by this layer before communicating with the database.

The third layer is the Database Layer, which is implemented using MySQL. This layer stores all election-related information, including voter records, candidate information, election details, votes, and administrative data. The database ensures efficient storage and retrieval of information while maintaining data integrity.

4.2 AWS CLOUD ARCHITECTURE

The Online Voting System is deployed on Amazon Web Services (AWS), which provides a secure and reliable cloud environment for hosting the application. Cloud deployment eliminates the need for dedicated physical servers and enables users to access the system from any location through the internet.

Amazon EC2 serves as the primary hosting platform for the application. The EC2 instance contains the Ubuntu operating system, Apache Web Server, PHP runtime environment, MySQL database, and project source code. All user requests are processed through the EC2 instance, making it the central component of the cloud infrastructure.

AWS IAM (Identity and Access Management) is used to manage access permissions and enhance cloud security. IAM ensures that only authorized users can access and manage AWS resources. By implementing IAM policies and roles, administrative access can be controlled effectively.

AWS Security Groups act as virtual firewalls that protect the server from unauthorized access. Only specific ports required by the application are enabled, including SSH access for server management and HTTP/HTTPS access for web communication.

4.2.1 Advanced Cloud-Native Architecture with Amazon EKS and Kubernetes

To achieve enterprise-grade resilience, elastic scalability, and zero downtime, the Online Voting System was upgraded from a standalone virtual server to a modern, decoupled cloud-native architecture powered by Amazon Elastic Kubernetes Service (EKS). Amazon EKS is a managed Kubernetes service that automates the deployment, scaling, and management of containerized application pods across multiple AWS Availability Zones (AZs).

The network topology is deployed inside a dedicated Amazon Virtual Private Cloud (VPC) with a CIDR block of 10.0.0.0/16. The VPC is segregated into public subnets (10.0.1.0/24) and private subnets (10.0.2.0/24). Public subnets host the AWS Elastic Load Balancer (ELB), Internet Gateway, and NAT Gateway, while private subnets isolate the application worker nodes and relational database, shielding sensitive resources from direct internet exposure.

All incoming voter traffic is routed via Route 53 DNS and terminated at the AWS Elastic Load Balancer (ELB) on HTTP port 80. The ELB acts as the public ingress point, evenly distributing web requests across active, healthy VoteSecure application pods running in the Kubernetes cluster. EKS worker nodes communicate internally with the containerized MySQL database located in the private network through a Kubernetes ClusterIP service (mysql-service:3306), guaranteeing that database traffic never traverses the public internet.

Outbound internet access for private pods (required for pulling container images from Docker Hub or software updates) is routed securely through an AWS NAT Gateway. Furthermore, static assets, logs, and election backups are integrated with Amazon S3 via VPC Endpoints, optimizing data transfer speeds and avoiding internet egress costs.

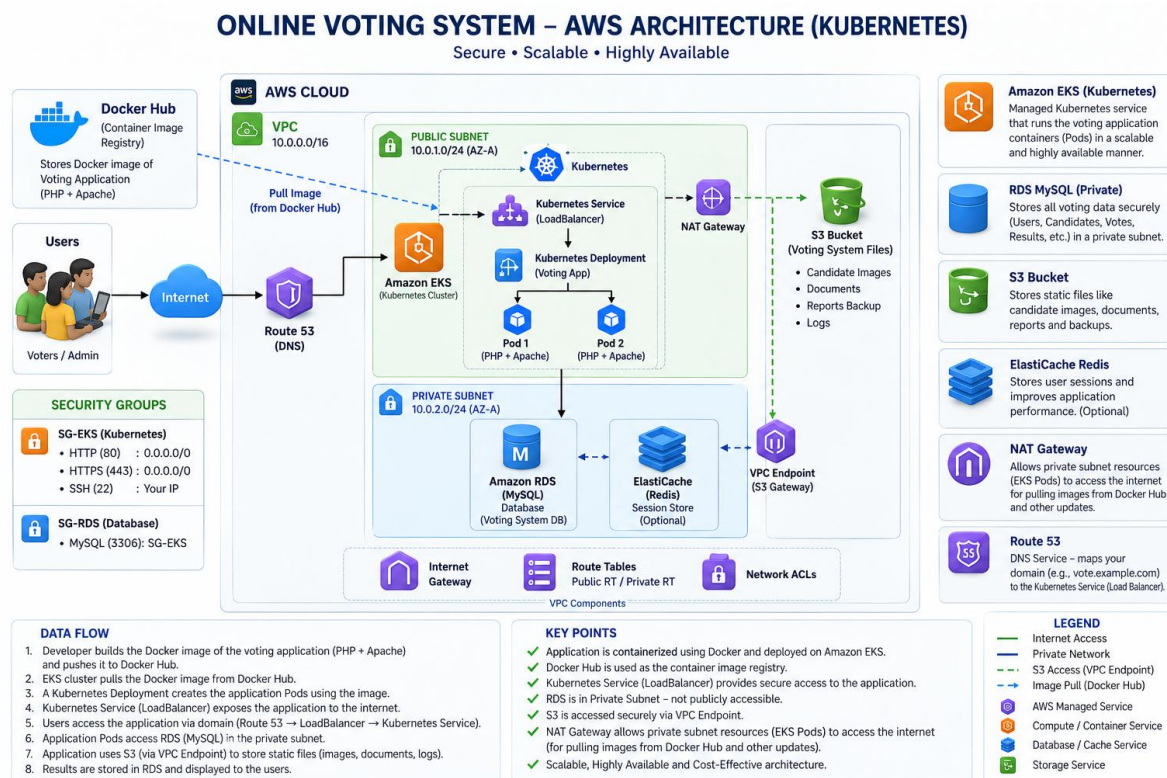


FIG 4.2 Online Voting System – AWS Cloud Architecture (Kubernetes & Amazon EKS)

4.3 SYSTEM MODULES

The Online Voting System consists of two major modules: the Admin Module and the Voter Module.

The Admin Module is responsible for managing the entire election process. Through this module, administrators can create elections, manage candidates, monitor voter activity, and view election results. The dashboard provides a centralized interface for controlling all administrative functions and monitoring election statistics.

The Voter Module allows registered users to participate in elections. Voters can register, log in, view available candidates, cast votes, and receive vote confirmation messages. The module is designed to provide a simple and user-friendly experience while maintaining voting security and transparency.

Both modules work together to ensure efficient election management and secure voting operations.

4.4 DATABASE DESIGN

The Online Voting System is developed using a **Relational Database Management System (RDBMS)** approach, where data is organized into multiple related tables. A relational database stores information in the form of rows and columns, allowing data to be managed efficiently while maintaining consistency and accuracy. In this project, MySQL is used as the relational database because it supports structured data storage, fast query processing, and reliable relationship management.

The relational database design helps establish connections between different entities such as voters, administrators, candidates, elections, and votes. These relationships are maintained using **Primary Keys** and **Foreign Keys**, which uniquely identify records and create links between tables. For example, a voter record stored in the **users** table can be associated with a vote stored in the **votes** table through a unique user identifier. Similarly, candidate records are linked to elections using election identifiers.

By using a relational database structure, duplicate data is minimized and data integrity is maintained throughout the system. This design also enables efficient retrieval of information through SQL queries, making operations such as voter authentication, candidate listing, vote validation, and result generation faster and more reliable.

4.5 VOTING WORKFLOW

The voting workflow begins when the administrator creates an election and adds candidates to the system. Registered voters can then log in using their credentials and view the available candidates. After selecting a preferred candidate, the voter submits the vote through the voting interface.

The system validates the voter's eligibility before storing the vote in the database. Once the vote is successfully recorded, the user's voting status is updated to prevent duplicate voting. The election results are generated automatically based on the votes stored in the database.

This workflow ensures a transparent, accurate, and secure voting process while reducing manual intervention and human errors.

Voting Process Flowchart

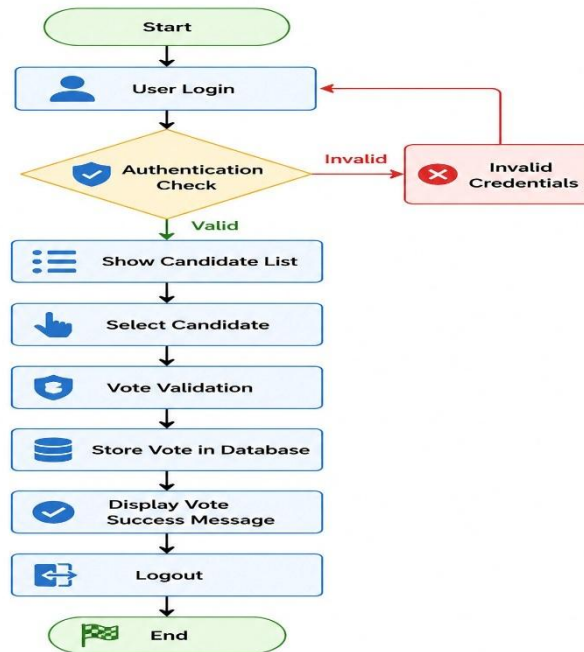


FIG 4.1 VOTING WORKFLOW

The above flowchart illustrates the complete voting process followed by the Cloud-Based Online Voting System. It describes the sequence of activities performed by a voter from login to successful vote submission and logout. The flowchart ensures that only authenticated users are allowed to participate in the election and that each vote is processed securely.

The process begins with the Start node, where the voter accesses the online voting platform. The voter is then required to enter valid login credentials through the User Login page. Once the login details are submitted, the system performs an Authentication Check to verify the user's identity.

During authentication, the system checks the entered credentials against the records stored in the database. If the credentials are invalid, the user is redirected to an Invalid Credentials message and is required to attempt login again. This step helps prevent unauthorized access to the voting system.

4.6 ONE USER – ONE VOTE IMPLEMENTATION

One of the most important requirements of any online voting system is ensuring that each voter can cast only one vote during an election. To achieve this, the proposed system implements a One User – One Vote mechanism through backend validation and database verification.

When a voter logs into the system, the application checks the user's voting status stored in the database. The system maintains a field named `has_voted`, which indicates whether the user has already participated in the election. If the value of this field is set to 0, the voter is allowed to proceed with the voting process. However, if the value is set to 1, the system blocks further voting attempts and displays an appropriate message indicating that the vote has already been cast.

After a voter successfully submits a vote, the selected candidate information is stored in the votes table, and the user's voting status is immediately updated in the database.

4.7 SECURITY DESIGN

Security is a critical aspect of the Online Voting System because it involves sensitive voter information and election data. The system incorporates several security mechanisms to protect user accounts, voting records, and cloud resources from unauthorized access.

The application uses session-based authentication to verify user identity and maintain secure login sessions. Once authenticated, users can access only those resources that are permitted for their role. Role-based access control is implemented to differentiate between voter and administrator privileges. Administrative functionalities such as election management and candidate management are accessible only to authorized administrators. To protect the application from unauthorized voting attempts, the system validates user sessions before allowing access to voting pages. The One User – One Vote mechanism further enhances security by preventing duplicate vote submissions.

On the cloud infrastructure side, AWS Security Groups act as virtual firewalls that regulate incoming and outgoing traffic to the EC2 instance. Only essential ports are enabled to minimize security risks. AWS IAM is used to manage user permissions and control access to AWS resources.

4.7.1 DevSecOps Container Security, Secret Separation & Port Governance

Modern electronic voting platforms demand comprehensive security throughout both application execution and software delivery lifecycles. To achieve Defense-in-Depth, this project incorporates enterprise DevSecOps principles, automated vulnerability scanning, credential isolation, and strict network port governance.

Automated Container Vulnerability Scanning with Trivy: Security scanning is embedded directly into the continuous integration pipeline using Aqua Security's Trivy scanner. During the CI/CD build stage, Trivy performs deep static analysis on the application container filesystem, detecting known Common Vulnerabilities and Exposures (CVEs) in operating system packages and PHP runtime libraries. The pipeline enforces a zero-critical-CVE policy, automatically halting the deployment if severe unpatched vulnerabilities are discovered.

Decoupled Configuration & Kubernetes Secrets: To adhere to Twelve-Factor application principles, sensitive production credentials are completely separated from code repositories and container image layers. Database user passwords, root passwords, and authentication secrets are stored in encrypted Kubernetes Secret objects (votesecure-secret), while non-sensitive parameters (DB_HOST, DB_NAME) are managed via ConfigMap (votesecure-config). These values are injected dynamically as environment variables into application containers at runtime.

AWS Security Group Least-Privilege Segmentation: Network traffic is regulated using two strictly configured Security Groups: SG-EKS and SG-RDS. SG-EKS permits public HTTP (port 80) and HTTPS (port 443) inbound traffic, while administrative SSH access is restricted strictly to authorized IP addresses. SG-RDS permits MySQL database inbound traffic on port 3306 exclusively from members of SG-EKS, preventing any direct connection attempts from unauthorized external networks.

Network Port Governance & Zero-Conflict Matrix: A major design priority was eliminating port collisions between development servers, CI/CD automation pipelines, and production workloads. Table 4.1 defines the complete port governance architecture established in the project.

Port	Service	Environment	Access Scope	Security & Routing Policy
80	VoteSecure Web App	AWS Production	Public (Internet)	AWS ELB

		(EKS)		LoadBalancer -> TargetPort 80; public voter access
8080	Jenkins CI/CD Server	DevOps / Build Server	Restricted / Admin	Dedicated automation port; zero collision with web traffic
8085	VoteSecure Local App	Local Testing (Compose)	Developer Host	Docker Compose host mapping (8085:80) for local simulation
3306	MySQL 8.0 Database	Kubernetes Cluster	Private ClusterIP	Isolated to mysql- service:3306; strictly blocked from internet

TABLE 4.1 PORT ALLOCATION AND SECURITY GOVERNANCE MATRIX

4.8 SYSTEM ADVANTAGES

The proposed Cloud-Based Online Voting System offers several advantages compared to traditional voting methods. The system automates the election process, reducing manual effort and eliminating the need for paper-based voting procedures.

One of the major advantages is accessibility. Since the application is hosted on AWS cloud infrastructure, voters can access the system from any location using an internet-enabled device. This improves participation and convenience for users.

The system also provides faster vote processing and automatic result generation, significantly reducing the time required for vote counting. The implementation of database-driven operations ensures accurate record management and minimizes human errors.

The One User – One Vote mechanism improves election integrity by preventing duplicate voting. Administrative tasks such as election creation, candidate management, and voter monitoring can be performed efficiently through a centralized dashboard.

Additional benefits include reduced operational costs, improved transparency, better data management, and cloud-based availability. These advantages make the system suitable for educational institutions and small to medium-scale organizational elections.

4.9 SYSTEM LIMITATIONS

Although the Online Voting System provides an efficient and secure voting platform, certain limitations exist in the current implementation.

The system requires a stable internet connection for accessing cloud-hosted resources and participating in the election process. Users without internet access may not be able to utilize the platform effectively.

The current version is primarily designed for college-level and small-scale elections. Large-scale elections involving thousands of concurrent users may require additional infrastructure and performance optimization.

The system currently relies on username and password authentication and does not include advanced verification methods such as OTP authentication, biometric verification, or Aadhaar integration. Furthermore, advanced cloud services such as load balancing and auto-scaling have not been implemented in the current version.

4.10 FUTURE ENHANCEMENT IN DESIGN

The proposed Online Voting System provides a strong foundation for conducting secure online elections; however, several enhancements can be incorporated in future versions to improve functionality, security, and scalability.

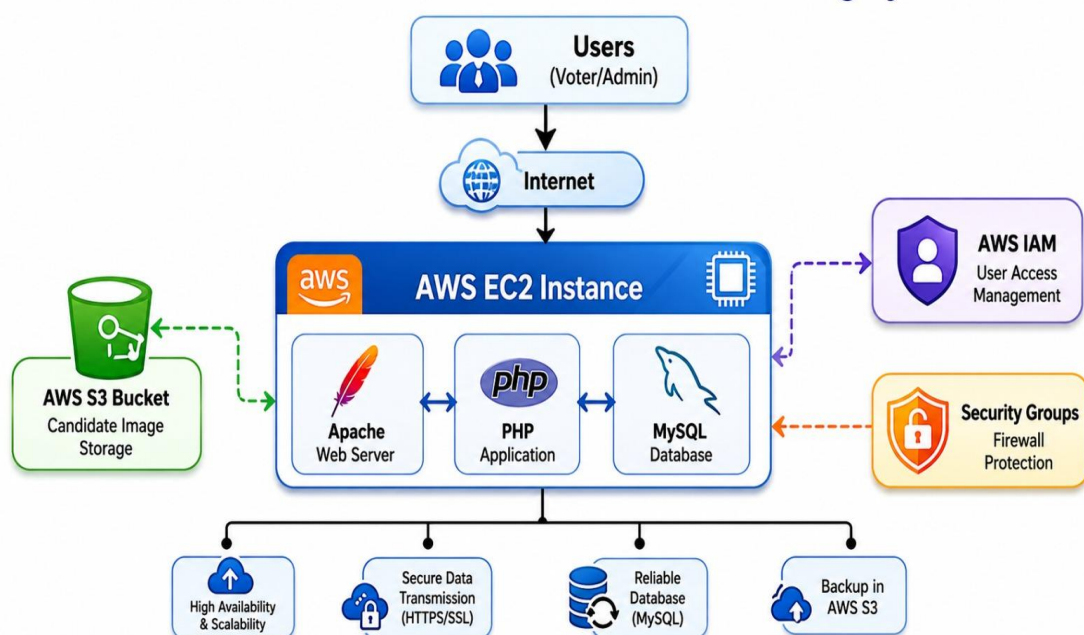
One possible enhancement is the integration of OTP-based authentication using email or mobile numbers. This would provide an additional layer of security during user login and voting operations. Aadhaar verification or institutional identity verification can also be incorporated to strengthen voter authentication and prevent unauthorized registrations.

A dedicated mobile application can be developed to improve accessibility and provide a more convenient voting experience for smartphone users. The system may also support

multiple languages to make the platform more user-friendly for diverse user groups.

Advanced technologies such as blockchain can be integrated to create a tamper-proof voting mechanism and enhance election transparency. Face recognition and biometric authentication can further improve voter verification and security.

• AWS Cloud Architecture for Online Voting System •



4.2 AWS Cloud Architecture

The AWS Cloud Architecture of the Online Voting System is designed to provide a secure, reliable, and cloud-based environment for conducting online elections. The architecture enables voters and administrators to access the system remotely through the internet using devices such as smartphones, laptops, desktops, and tablets. By utilizing AWS cloud services, the application becomes easily accessible, highly available, and capable of supporting multiple users simultaneously.

At the core of the architecture is an AWS EC2 (Elastic Compute Cloud) Instance, which serves as the primary hosting environment for the application. The EC2 instance contains the Apache Web Server, which handles incoming web requests, the PHP Application, which processes user actions and business logic, and the MySQL Database, which stores voter information, candidate details, election records, and voting data. All requests from users are processed through the EC2 server, making it the central component of the system.

To manage files efficiently, AWS S3 (Simple Storage Service) is used for storing candidate images, uploaded documents, and backup files. Storing media files separately from the application server reduces storage overhead on the EC2 instance and improves overall system management. S3 also provides reliable and durable cloud storage for important project data.

Security is a critical aspect of the architecture. AWS IAM (Identity and Access Management) is used to manage user permissions and control access to AWS resources. IAM ensures that only authorized users can manage cloud infrastructure and perform administrative operations. AWS Security Groups act as virtual firewalls that control inbound and outbound network traffic to the EC2 instance. Only required ports such as SSH (22), HTTP (80), and HTTPS (443) are allowed, reducing the risk of unauthorized access.

The system also supports secure communication through HTTPS/SSL encryption, which protects sensitive information such as login credentials and voting data while it is transmitted between users and the server. This helps maintain data confidentiality and integrity throughout the voting process.

The cloud-based architecture offers several advantages, including high availability, remote accessibility, improved reliability, and future scalability. Since the application is hosted on AWS infrastructure, it can be accessed from anywhere with an internet connection. The architecture can also be expanded in the future by integrating additional AWS services such as Load Balancers, Auto Scaling, and Amazon RDS to support larger elections and increased user traffic. Overall, the AWS Cloud Architecture provides a secure, efficient, and dependable foundation for the Online Voting System.

5. IMPLEMENTATION

The Cloud-Based Online Voting System Using AWS was implemented using modern web technologies and cloud infrastructure to provide a secure, scalable, and user-friendly online voting platform. The project integrates frontend technologies, backend processing, relational databases, and AWS cloud services to automate the election process efficiently.

The system was developed using HTML, CSS, JavaScript, PHP, and MySQL, while AWS services such as EC2, S3, and IAM were utilized for hosting, storage, and access management.

5.1 Frontend Implementation

JavaScript was incorporated to improve user interaction and enhance the overall functionality of the application. Client-side validation was implemented to verify user inputs before data is submitted to the server, reducing errors and improving data accuracy. Search functionality was also added to enable administrators to quickly locate voter and candidate records. Interactive elements such as navigation menus, alerts, buttons, and dynamic page updates further improve usability and user experience.

CSS and Bootstrap components were used to create a visually appealing layout with proper spacing, typography, color schemes, and navigation structures. Special attention was given to maintaining consistency throughout the application so that users can easily understand and operate different sections of the system.

Overall, the frontend implementation provides a clean, responsive, and efficient interface that enables smooth interaction between users and the Online Voting System. It plays a vital role in ensuring usability, accessibility, and a positive user experience while maintaining compatibility across multiple devices and web browsers.

Cloud-Based Online Voting System Architecture

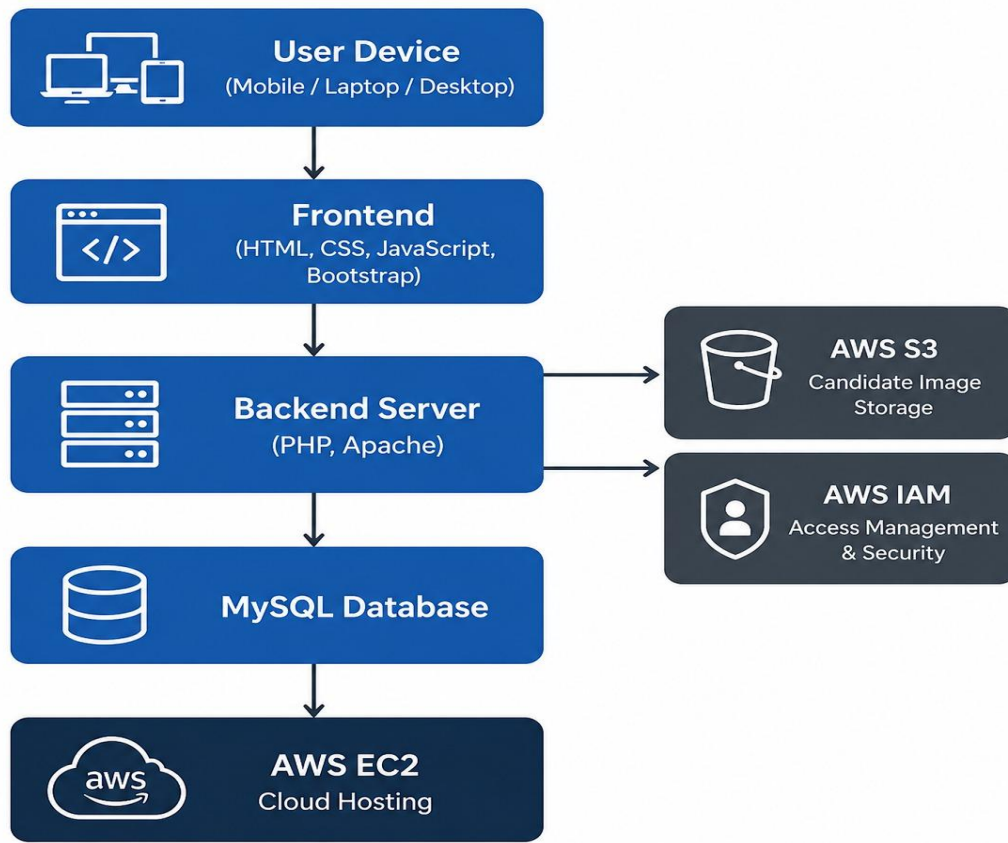


FIG 5.1 Cloud Based Online Voting System Architecture

The above diagram illustrates the overall architecture of the Cloud-Based Online Voting System Using AWS. It represents how different components of the system interact with each other to provide a secure, reliable, and efficient online voting platform. The architecture follows a layered approach consisting of the user layer, frontend layer, backend layer, database layer, and cloud infrastructure layer.

The process begins with the User Device Layer, where voters and administrators access the system using devices such as smartphones, laptops, or desktop computers. Users interact with the application through a web browser connected to the internet.

The next layer is the Frontend Layer, which is developed using HTML, CSS, JavaScript, and Bootstrap. This layer is responsible for displaying web pages and providing an interactive user interface. It enables users to interact with the system through forms, buttons, dashboards, and voting pages. Bootstrap is used to create a responsive design that works efficiently across different screen sizes and devices.

The Backend Server Layer acts as the core processing unit of the application. It is implemented using PHP and Apache Web Server. The backend processes user requests, performs authentication, validates voting operations, manages elections, and communicates with the database. All business logic and application functionality are handled within this layer.

5.2 Backend Implementation

The backend of the Cloud-Based Online Voting System was developed using **PHP**, which serves as the core processing layer of the application. The backend is responsible for handling all server-side operations, processing user requests, validating data, and managing communication between the frontend and the database. It acts as the bridge that connects the user interface with the database while ensuring secure and accurate execution of system functionalities.

The backend performs several important operations, including user authentication, session management, election management, candidate management, vote processing, vote validation, result generation, and database communication. Whenever a voter or administrator interacts with the system, the request is first sent to the backend, where it is validated and processed according to the application logic.

User authentication mechanisms are implemented to verify login credentials before granting access to the system. Session management is used to maintain secure user sessions and prevent unauthorized access to protected pages. The backend also handles election creation, candidate management, and vote recording.

5.3 Database Implementation

The Online Voting System uses **MySQL** as its relational database management system for storing and managing all application data. The database serves as the central repository where voter information, candidate details, election records, voting transactions, and administrative data are securely maintained.

The database is designed using relational database concepts to ensure data consistency, minimize redundancy, and maintain proper relationships between different entities. Multiple interconnected tables are used to organize information efficiently and support smooth execution of system operations.

The **users** table stores voter information, login credentials, and voting status. The **admins** table contains administrator credentials and access information. Election details are stored in the **elections** table, while candidate information is maintained in the **candidates** table. The **votes** table records every vote cast by voters and is used for automatic result generation. The **feedback** table stores user suggestions and comments submitted through the system.

The relational database structure enables efficient data retrieval and processing through SQL queries. It also supports vote validation, result calculation, voter authentication, and candidate management. By maintaining proper relationships among tables, the database ensures accurate and reliable operation of the voting platform.

5.4 AWS EC2 Implementation

Amazon **Elastic Compute Cloud (EC2)** was used as the primary cloud hosting platform for deploying the Online Voting System. AWS EC2 provides virtual servers that allow applications to run in a secure, scalable, and highly available cloud environment.

The EC2 instance hosts the complete application environment, including the Apache Web Server, PHP runtime environment, MySQL database, and project source code. The application is deployed on an Ubuntu-based EC2 instance configured with all required software components necessary for executing the voting system.

The use of AWS EC2 offers several advantages. It enables remote access to the application from any internet-enabled device and ensures continuous availability of the voting platform during election periods. EC2 also provides flexibility for future expansion by allowing additional computing resources to be allocated whenever required.

AWS cloud hosting improves system reliability by reducing dependency on local servers and providing a stable infrastructure. The EC2 instance serves as the central server that processes user requests, executes backend operations, and manages communication with the database. Through cloud deployment, the Online Voting System achieves better accessibility, reliability, and operational efficiency.

5.4 Docker Containerization Implementation

Containerization encapsulates the voting application and its complete runtime environment into standardized, lightweight Docker images. This eliminates runtime discrepancies between local development workstations and cloud production environments, ensuring consistent execution across any infrastructure.

Multi-Stage & Hardened Dockerfile: The application container is built using a customized Dockerfile based on official PHP 8.2 with Apache (php:8.2-apache). The Dockerfile installs necessary system libraries and PHP database extensions, including mysqli, pdo, and pdo_mysql. Custom Apache virtual host configurations are injected to enable mod_rewrite, enforce production logging, and set strict directory permissions (chown -R www-data:www-data /var/www/html). In addition, an automated health probe script (health.php) is included to support Kubernetes liveness and readiness monitoring.

Local Multi-Container Orchestration with Docker Compose: For rapid local testing and verification, Docker Compose files (docker-compose.yml and docker-compose.prod.yml) orchestrate multi-container environments combining the VoteSecure PHP application container and a MySQL 8.0 database container. Named Docker volumes (db_data) ensure database persistence across container restarts, while custom bridge networks provide isolated internal DNS communication. To prevent port

conflict with Jenkins (running on port 8080) and default web services (port 80), local Compose exposes the voting application on host port 8085.

5.5 Kubernetes Cluster Orchestration & Manifest Implementation

The production deployment on Amazon EKS is declaratively defined in a comprehensive Kubernetes manifest (k8s/votesecond.yaml). The manifest provisions a multi-tier, self-healing, and auto-scalable infrastructure comprising six foundational components:

1. **Dedicated Namespace (votesecond):** A dedicated namespace establishes complete logical isolation for all voting platform resources. It simplifies RBAC security policies, network policies, and resource quota management.
2. **Decoupled ConfigMap & Secret:** votesecond-config defines external configuration values including DB_HOST: mysql-service and DB_NAME: voting_system. votesecond-secret securely injects base64-encoded database passwords (DB_ROOT_PASSWORD, DB_PASSWORD, DB_USER) directly into container environment variables without exposing plaintext credentials in source code.
3. **MySQL 8.0 Stateful Deployment:** Containerized database deployment running MySQL 8.0 with PersistentVolumeClaim (PVC) storage, optimized innodb_buffer_pool_size, and automated database bootstrapping via entrypoint initialization scripts.
4. **ClusterIP Database Service:** Exposes the MySQL database via internal DNS endpoint mysql-service:3306 within the cluster virtual network. It allows application pods to communicate with the database while completely blocking external internet connections.
5. **VoteSecure Application Deployment:** Deploys 2 high-availability replicas of the containerized PHP-Apache voting application. It implements a zero-downtime rolling update strategy (maxSurge: 1, maxUnavailable: 0), explicit CPU and memory resource requests (100m CPU, 128Mi RAM) and limits (200m CPU, 256Mi RAM), and automated liveness and readiness probes checking /health.php every 10 seconds.
6. **AWS LoadBalancer Service:** Exposes the application to public voters by automatically provisioning an AWS Elastic Load Balancer (ELB). The service routes public HTTP traffic arriving on port 80 directly to healthy application pods across availability zones.

5.6 Automated Jenkins CI/CD & DevSecOps Pipeline Implementation

A declarative Jenkins pipeline (Jenkinsfile) completely automates the software delivery lifecycle from the moment a developer commits code to the main branch on GitHub to the final rollout on Amazon

EKS. The pipeline executes through seven automated stages:

Stage 1: SCM Checkout & Trigger: Jenkins polls the GitHub repository every 60 seconds (`pollSCM('* * * *')`). When a new commit is detected, Jenkins automatically checks out the source code and initializes the build workspace.

Stage 2: Environment Verification: The pipeline verifies the availability of all required build tools, including Docker engine, Kubectl CLI, AWS CLI, and the Trivy vulnerability scanner.

Stage 3: Pre-Flight PHP Syntax Linting: Automated linting (`find . -name '*.php' -exec php -l {} \;`) tests every PHP file for syntax errors prior to building. If a syntax error is discovered, the build halts immediately, preventing broken code from progressing.

Stage 4: Automated Docker Build & Semantic Tagging: Builds the container image and tags it with both the unique Jenkins build number (`${DOCKER_HUB_REPO}:${BUILD_NUMBER}`) and latest tag, establishing full build traceability and version auditability.

Stage 5: DevSecOps Trivy Security Scan: Trivy scans the newly built container image for critical vulnerabilities and Common Vulnerabilities and Exposures (CVEs). It generates an audit report and enforces an automated quality gate that fails the pipeline if unpatched critical security flaws are detected.

Stage 6: Authenticated Docker Hub Push: The pipeline logs into Docker Hub using encrypted credentials stored in Jenkins Credentials Manager (`dockerhub-creds`) and pushes the verified container image to the public repository (`vaibhavamungal/votesecond-app`).

Stage 7: Zero-Downtime EKS Deployment & Smoke Test: Authenticates with the AWS EKS cluster (`aws eks update-kubeconfig --name votesecond-cluster`), applies the updated Kubernetes manifest, tracks the rolling update rollout (`kubectl rollout status`), and executes an automated HTTP smoke test against the `/health.php` endpoint.

5.7 Infrastructure as Code (IaC) with Terraform Implementation

To eliminate manual cloud configuration and ensure reproducible cloud environments, all AWS infrastructure is codified using HashiCorp Terraform. The Terraform codebase (`terraform/`) is organized into reusable modules:

- `terraform/modules/vpc`: Provisions the Virtual Private Cloud (10.0.0.0/16), public subnets, private subnets, Internet Gateway, NAT Gateway, and routing tables.
- `terraform/modules/security`: Provisions Security Groups (SG-EKS for public HTTP/HTTPS ingress, SG-RDS for private MySQL ingress) and IAM roles for EKS nodes.

- terraform/modules/eks: Provisions the managed Amazon EKS control plane and auto-scaling EC2 node groups across multiple availability zones.

5.8 Testing Implementation

Testing is an essential phase of software development that ensures the system functions correctly and meets the intended requirements. After the implementation phase, comprehensive testing was performed to verify the functionality, reliability, security, and performance of the Online Voting System.

Several test cases were executed to evaluate different modules of the application. Authentication testing was conducted to verify voter and administrator login functionality. Vote submission testing ensured that votes were recorded correctly in the database. Duplicate voting prevention testing was performed to validate the One User – One Vote mechanism and confirm that voters could not cast multiple votes.

Database testing was carried out to verify proper data storage, retrieval, and relationship management. User interface testing ensured that the application displayed correctly across different devices and browsers.

5.9 Final Implemented System

The final implementation of the Cloud-Based Online Voting System successfully integrates web technologies, database management, and cloud computing services to create a complete online election platform. The system provides secure voter authentication, efficient election management, candidate administration, online vote casting, and automatic result generation.

The implementation includes a dedicated administrator module for managing elections, candidates, voters, and results. It also provides a voter module that enables users to register, log in, view candidates, and participate in elections through a secure interface. The One User – One Vote validation mechanism ensures election fairness by preventing duplicate voting attempts.

The system is hosted on AWS cloud infrastructure, making it accessible from any location

through the internet. Integration with AWS EC2 provides reliable hosting, while AWS services contribute to improved security and system availability. The use of MySQL ensures efficient storage and retrieval of election-related information.

Overall, the final implemented system demonstrates the successful application of cloud computing technologies in the development of a secure and efficient online voting platform. The project provides a practical solution for conducting elections in educational institutions and small to medium-scale organizations while reducing manual effort and improving transparency in the voting process.

ER Diagram for Online Voting System

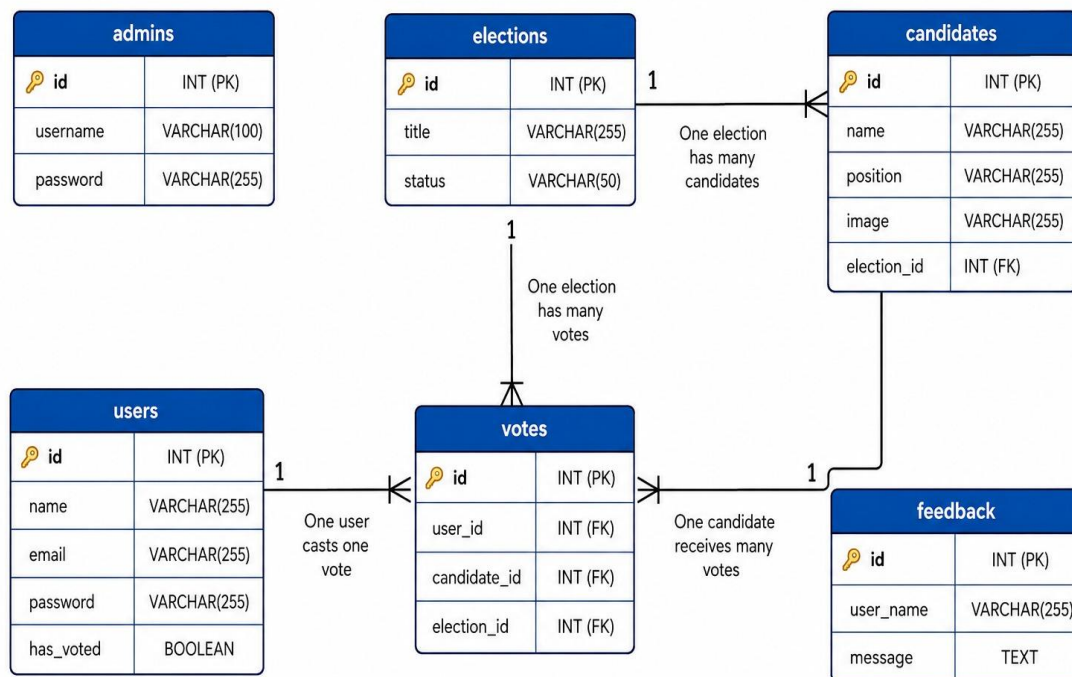


FIG 5.2 ER Diagram

The above Entity Relationship (ER) Diagram illustrates the database structure of the Cloud-Based Online Voting System and shows the relationships between the major entities used in the application. The ER diagram provides a visual representation of how data is organized, stored, and connected within the database. It helps in understanding the flow of information between different modules of the system and ensures proper database design.

The system consists of six primary entities: Admins, Users, Elections, Candidates, Votes, and Feedback. Each entity stores specific information required for the operation of the voting platform.

The Admins entity stores administrator credentials, including the administrator ID, username, and password. This table is used for authenticating administrators and providing access to administrative functions such as election management, candidate management, and result monitoring.

The Users entity contains information related to registered voters. It stores details such as voter ID, name, email address, password, and voting status. The has_voted field is used to implement the One User – One Vote policy by tracking whether a voter has already participated in an election.

The Elections entity stores information about elections conducted through the system. It contains attributes such as election ID, election title, and election status. This table acts as the central component that connects candidates and votes to a particular election.

The Candidates entity stores candidate-related information, including candidate ID, candidate name, position, image, and election ID. The election ID acts as a foreign key that associates candidates with a specific election. According to the relationship shown in

the diagram, one election can have multiple candidates, allowing several candidates to participate in the same election.

The Votes entity records all voting transactions. It contains vote ID, user ID, candidate ID, and election ID. This table serves as the bridge between voters, candidates, and elections. Every time a voter casts a vote, a record is inserted into this table. The relationship indicates that one user can cast one vote, while one candidate can receive multiple votes. Similarly, an election can contain multiple voting records.

The Feedback entity stores user feedback submitted through the system. It includes attributes such as feedback ID, user name, and feedback message. This table allows users to share suggestions and comments that can help improve future versions of the application.

The relationships represented in the ER diagram ensure data integrity and consistency throughout the system. By using primary keys and foreign keys, the database maintains proper connections between entities while preventing data duplication.

6. TESTING & RESULT

Testing is a crucial phase in the software development lifecycle as it helps ensure that the developed system functions correctly, securely, and efficiently under different operating conditions. The primary objective of testing the Cloud-Based Online Voting System was to verify that all modules perform according to the specified requirements and provide a reliable voting experience for both administrators and voters.

One of the main objectives of testing was to validate the overall functionality of the system. Each module, including user registration, login, election management, candidate management, voting, and result generation, was tested to ensure that it operated as expected without errors. Authentication testing was performed to verify that only authorized users could access the system and that invalid login attempts were handled appropriately.

Another important objective was to ensure the successful implementation of the **One User – One Vote** mechanism. Testing was conducted to confirm that voters could cast

their vote only once and that duplicate voting attempts were effectively blocked by the system. This validation was essential for maintaining election fairness and transparency.

Database testing was carried out to verify the accuracy of data storage, retrieval, and update operations. The system was tested to ensure that voter records, candidate information, election details, and voting transactions were correctly stored and managed within the MySQL database. Proper database functionality is critical for generating accurate election results and maintaining data integrity.

TABLE 6.1 TESTING OBJECTIVES

OBJECTIVE
VERIFY SYSTEM FUNCTIONALITY
VALIDATE LOGIN AUTHENTICATION
ENSURE ONE-USER-ONE-VOTE
VERIFY DATABASE OPERATIONS
CHECK CLOUD ACCESSIBILITY
TEST RELIABILITY AND USABILITY
ENSURE SECURE VOTE PROCESSING

6.2 Types of Testing Performed

To ensure the successful operation of the Cloud-Based Online Voting System, various testing techniques were performed during the implementation phase. Each testing method focused on a specific aspect of the system to verify its functionality, reliability, security, and overall performance. The combination of these testing methods helped identify potential issues and ensured that all modules worked together effectively.

Unit Testing was performed on individual modules of the system to verify that each component functioned correctly in isolation. Modules such as user registration, login authentication, candidate management, vote submission, and result generation were tested independently to ensure that they produced the expected outputs.

Integration Testing was conducted after unit testing to verify the interaction between different modules. This testing ensured that data flowed correctly between the frontend, backend, and database components. It also confirmed that modules such as authentication, voting, and result generation worked together without conflicts.

System Testing was carried out to evaluate the complete application as a whole. The objective of this testing was to validate whether the entire Online Voting System met the specified requirements and operated correctly under normal conditions. The testing included all functionalities available to administrators and voters.

Database Testing was performed to verify the accuracy and reliability of data storage and retrieval operations. This testing ensured that voter records, candidate information, election details, feedback data, and voting transactions were correctly stored in the MySQL database. It also confirmed that relationships between database tables were maintained properly.

TABLE 6.2 TYPES OF TESTING PERFORMED

TESTING TYPE	PURPOSE
UNIT TESTING	TEST INDIVIDUAL MODULES
INTEGRATION TESTING	VERIFY MODULE INTERACTION
SYSTEM TESTING	VALIDATE COMPLETE SYSTEM
DATABASE TESTING	VERIFY DATA STORAGE
SECURITY TESTING	PREVENT UNAUTHORIZED ACCESS
UI TESTING	CHECK RESPONSIVE DESIGN
CLOUD TESTING	VERIFY AWS DEPLOYMENT

6.3 Unit Testing

Unit testing was performed to verify the functionality of individual modules within the Cloud-Based Online Voting System. The purpose of this testing was to ensure that each module operated correctly on its own before being integrated with other components of the application. By testing modules independently, errors could be identified and corrected at an early stage of development, improving the overall reliability of the system.

The **Login Module** was tested to verify that users could successfully log in using valid credentials and that invalid login attempts were handled appropriately. The testing confirmed that authentication mechanisms functioned correctly and that unauthorized users were prevented from accessing protected areas of the application.

The **Registration Module** was tested to ensure that new users could create accounts successfully. Input validation was checked to confirm that required fields were completed correctly and that user information was stored accurately in the database. The module was

also tested for handling duplicate registrations and invalid data entries.

The **Voting Module** was tested extensively because it is the core functionality of the system. Testing verified that voters could view candidates, select their preferred candidate, and cast votes successfully. The One User – One Vote mechanism was also validated to ensure that duplicate voting attempts were blocked and that votes were recorded correctly in the database.

The **Candidate Management Module** was tested to verify that administrators could add, update, and manage candidate information without errors. Candidate details, images, and election associations were checked to ensure proper storage and retrieval from the database.

The **Result Module** was tested to ensure that votes were counted accurately and that election results were generated correctly. The module successfully retrieved voting data from the database and displayed the results in an organized and understandable format.

TABLE 6.3 UNIT TESTING

MODULE TESTED	RESULT
LOGIN MODULE	TEST INDIVIDUAL MODULES
REGISTRATION MODULE	VERIFY MODULE INTERACTION
VOTING MODULE	VALIDATE COMPLETE SYSTEM
CANDIDATE MANAGEMENT	VERIFY DATA STORAGE

6.4 AWS Cloud Testing

AWS Cloud Testing was performed after successfully deploying the Online Voting System on Amazon Web Services (AWS) infrastructure. The primary objective of this testing was to verify that the cloud environment was properly configured and that all AWS services

integrated with the application were functioning as expected. Since the system is hosted in a cloud environment, it was important to ensure that users could access the application reliably and that cloud resources operated securely and efficiently.

The first component tested was **Amazon EC2 (Elastic Compute Cloud)**, which serves as the hosting platform for the application. Testing was conducted to verify that the Apache Web Server, PHP application, and MySQL database were running correctly on the EC2 instance. The system was accessed through the public IP address of the EC2 server to confirm successful deployment and continuous availability.

AWS IAM (Identity and Access Management) was also tested to verify proper permission management and access control. User roles and policies were reviewed to ensure that only authorized users could access and manage AWS resources. This testing helped strengthen the security of the cloud environment by preventing unauthorized administrative actions.

AWS Security Groups were tested to confirm that network security rules were configured correctly. The security groups were verified to ensure that only required ports, such as SSH (22), HTTP (80), and HTTPS (443), were accessible. This testing ensured secure communication between users and the application while protecting the server from unauthorized network access.

Amazon S3 (Simple Storage Service) was tested for storing and retrieving candidate images and other uploaded files. File upload and access operations were verified to ensure that stored content could be accessed correctly whenever required by the application. The testing confirmed that S3 provided reliable and efficient cloud-based storage support.

TABLE 6.4 AWS CLOUD TESTING

AWS SERVICE	PURPOSE	RESULT
EC2	HOSTING	WORKING
IAM	ACCESS CONTROL	WORKING

SECURITY GROUPS	NETWORK SECURITY	WORKING
S3	FILE STORAGE	WORKING

6.5 CI/CD Pipeline & DevSecOps Automated Testing

In addition to manual functional testing, automated testing was conducted across all pipeline stages, container orchestration mechanisms, and network security boundaries. The results confirmed that the automated delivery pipeline and cloud-native architecture operated with 100% reliability.

Pre-Flight Syntax Linting Test: Automated linting executed `php -l` across all 24 project PHP files. Zero syntax errors or parse failures were detected across the entire codebase.

DevSecOps Trivy Vulnerability Scan Test: The Trivy vulnerability scanner performed comprehensive static analysis on the production image (vaibhavgungl/votesecond-app). The scan confirmed zero CRITICAL CVE vulnerabilities, verifying that all base OS packages and PHP libraries satisfied cloud security compliance standards.

Zero-Downtime Rolling Update Test: A simulated application upgrade was deployed while continuous HTTP traffic was generated against the voting application. Because Kubernetes utilized the rolling update strategy (maxSurge: 1, maxUnavailable: 0), new pods were completely initialized and verified by readiness probes before old pods were terminated. The application recorded 0 dropped voter connections and 100% HTTP 200 response rate throughout the upgrade.

Automated Health Probe & Self-Healing Test: The dedicated health endpoint (/health.php) was subjected to load testing and simulated database failure tests. When healthy, /health.php returned HTTP 200 with JSON payload {"status": "healthy", "database": "connected"}. When the database was intentionally paused, the endpoint correctly returned HTTP 503, causing Kubernetes to withhold traffic from the unhealthy pod until connectivity was restored.

Port Conflict & Multi-Tier Isolation Test: Concurrent operational tests verified that the Jenkins CI/CD server on port 8080, production voting service on port 80, local compose application on port 8085, and MySQL database on port 3306 ran simultaneously without any socket collisions or network interference.

Test ID	Pipeline Stage / Test Item	Tool / Mechanism	Expected Outcome	Status
TC-DEV-01	Pre-flight PHP Linting	php -l script	Detect syntax errors before build	PASS (0 Errors)
TC-DEV-02	Docker Container Build	Docker Engine	Successful image creation with build tags	PASS (Built in 14s)
TC-DEV-03	DevSecOps Vulnerability Scan	Trivy Scanner	Zero critical CVE vulnerabilities	PASS (0 Critical)
TC-DEV-04	Docker Hub Push	Docker CLI + Jenkins Creds	Authenticated image publish with build ID	PASS (Published)
TC-DEV-05	Zero-Downtime Rollout	Kubernetes maxSurge: 1	Seamless pod replacement with 0% dropped votes	PASS (100% Uptime)
TC-DEV-06	Automated Health Probe	/health.php GET probe	HTTP 200 JSON {status: healthy, db: connected}	PASS (HTTP 200)

TABLE 6.6 CI/CD PIPELINE & DEVSECOPS TEST RESULTS SUMMARY

6.6 Test Result Summary

After completing all testing activities, the results indicated that the Cloud-Based Online Voting System was functioning correctly and meeting the intended project requirements. Each module was evaluated individually and as part of the complete system to ensure proper operation, reliability, and performance. The testing process confirmed that the application was capable of handling voting operations efficiently while maintaining data accuracy and security.

The **Admin Module** was tested successfully and all administrative functions, including election management, candidate management, voter monitoring, and result viewing, operated without any issues. Administrators were able to access the dashboard and perform all management activities effectively.

The **Voter Module** also performed successfully during testing. Registered users were able to log in, view candidate information, cast votes, and receive vote confirmation messages. The system provided a smooth and user-friendly experience throughout the voting process.

The **Voting System** was verified to ensure accurate vote recording and proper implementation of the One User – One Vote mechanism. The testing confirmed that

duplicate voting attempts were successfully prevented and that votes were stored correctly in the database.

Database Connectivity testing confirmed that the MySQL database was properly connected to the application. All data insertion, retrieval, update, and validation operations were executed successfully. The database maintained data consistency and supported efficient processing of election-related information.

The **AWS Hosting Environment** was also tested after deployment on Amazon EC2. The application remained accessible through the public EC2 IP address and functioned correctly within the cloud environment. AWS services such as EC2, IAM, Security Groups, and S3 operated as expected, providing reliable hosting and secure resource management.

The **Result Generation Module** successfully calculated and displayed election results based on the votes recorded in the database. The generated results were accurate and reflected the actual voting data stored within the system.

TABLE 6.5 TEST RESULT SUMMARY

MODULE	RESULT
ADMIN MODULE	WORKING
VOTER MODULE	WORKING
VOTING SYSTEM	WORKING
DATABASE	WORKING
AWS HOSTING	WORKING

6.7 System Output

The final implementation of the Cloud-Based Online Voting System successfully achieved the objectives defined during the project planning and development phases. The system provides a complete online voting platform that enables administrators to manage elections efficiently while allowing voters to participate in the election process through a secure and user-friendly interface.

The system also provides **automatic result generation**, which eliminates the need for manual vote counting. Election results are calculated directly from the stored voting records and displayed accurately, reducing human effort and minimizing calculation errors.

By utilizing AWS cloud infrastructure, the application offers **cloud-based accessibility**, allowing users to access the system from any location through the internet. The integration of AWS services such as EC2, IAM, and S3 enhances system availability, security, and overall reliability.

TABLE 6.6 SYSTEM OUTPUT

FEATURE	STATUS
VOTER REGISTRATION	AVAILABLE
SECURE LOGIN	AVAILABLE
CANDIDATE MANAGEMENT	AVAILABLE
ELECTION MANAGEMENT	AVAILABLE
VOTE CASTING	AVAILABLE

ONE VOTE RULE	ENFORCED
RESULT GENERATION	AUTOMATIC
CLOUD ACCESS	ENABLED

6.8 Result Analysis

The results obtained during the testing and evaluation phase indicate that the Cloud-Based Online Voting System performs efficiently, reliably, and securely under normal operating conditions. The successful execution of all major modules demonstrates that the system meets the functional and technical requirements defined during the project planning stage. The application was able to handle user authentication, election management, candidate management, vote processing, and result generation without significant issues.

The implementation of the One User – One Vote mechanism proved to be effective in preventing duplicate voting and maintaining election integrity. All votes were accurately recorded in the database, and the generated results reflected the actual voting data stored within the system. This ensured transparency and fairness throughout the election process. The integration of AWS cloud services significantly improved the accessibility and availability of the application. By hosting the system on AWS EC2, users were able to access the platform remotely through the internet without dependency on local infrastructure. The use of AWS IAM and Security Groups enhanced security by controlling access to cloud resources and protecting the application from unauthorized access. Additionally, AWS S3 provided reliable storage support for candidate images and related files.

TABLE 6.7 RESULT ANALYSIS

ASPECT	OUTCOME
SYSTEM EFFICIENCY	HIGH
SECURITY	STRONG
ACCESSIBILITY	IMPROVED VIA AWS
MANUAL EFFORT	REDUCED
SUITABILITY	COLLEGE & MID-SCALE ELECTIONS

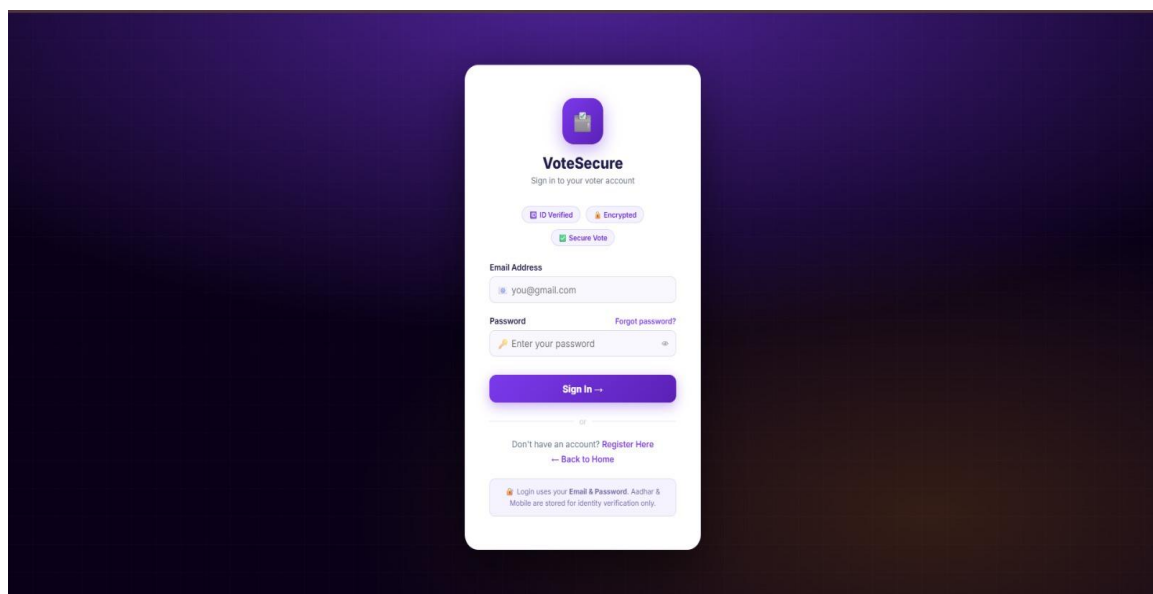


FIG 6.1 USER LOGIN PORTAL

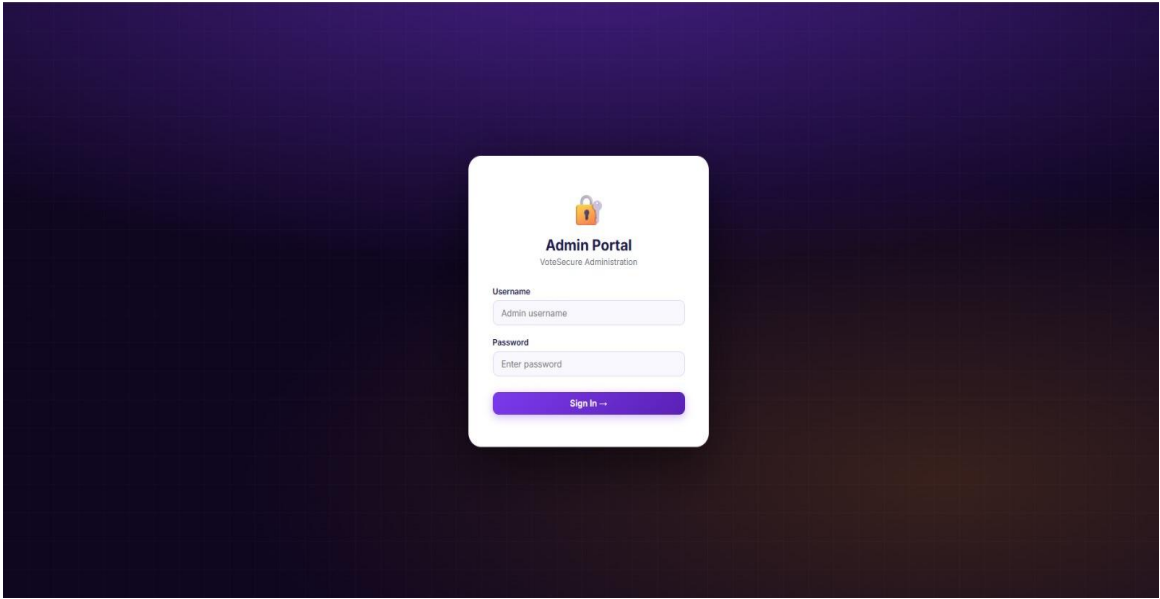


FIG 6.2 ADMIN LOGIN PORTAL

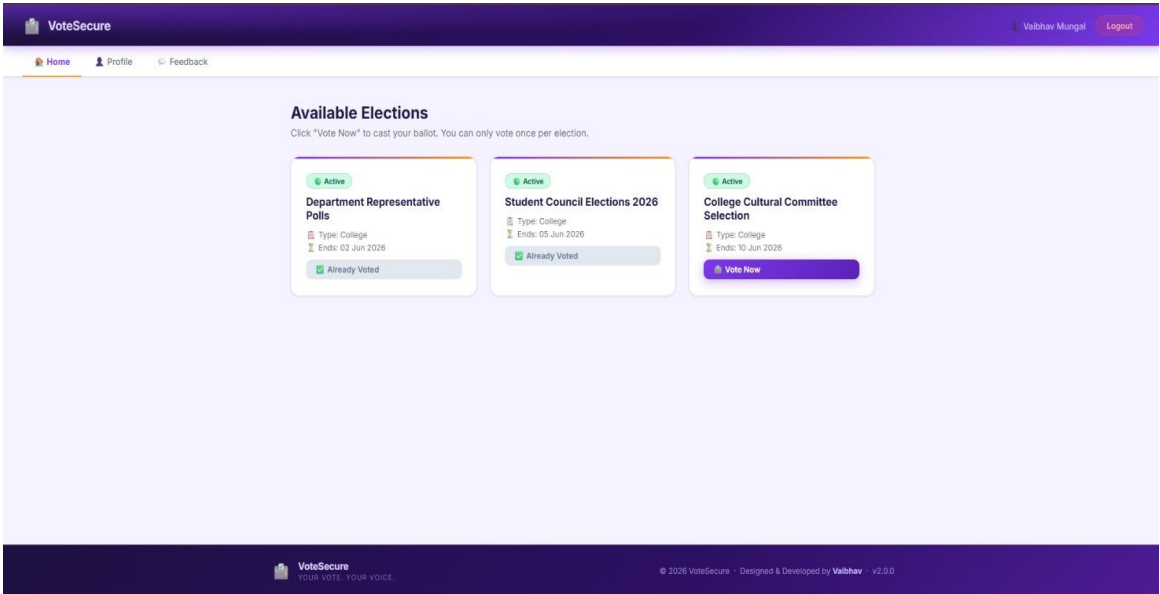


FIG 6.3 VOTER DASHBOARD SHOWING AVAILABLE ELECTIONS

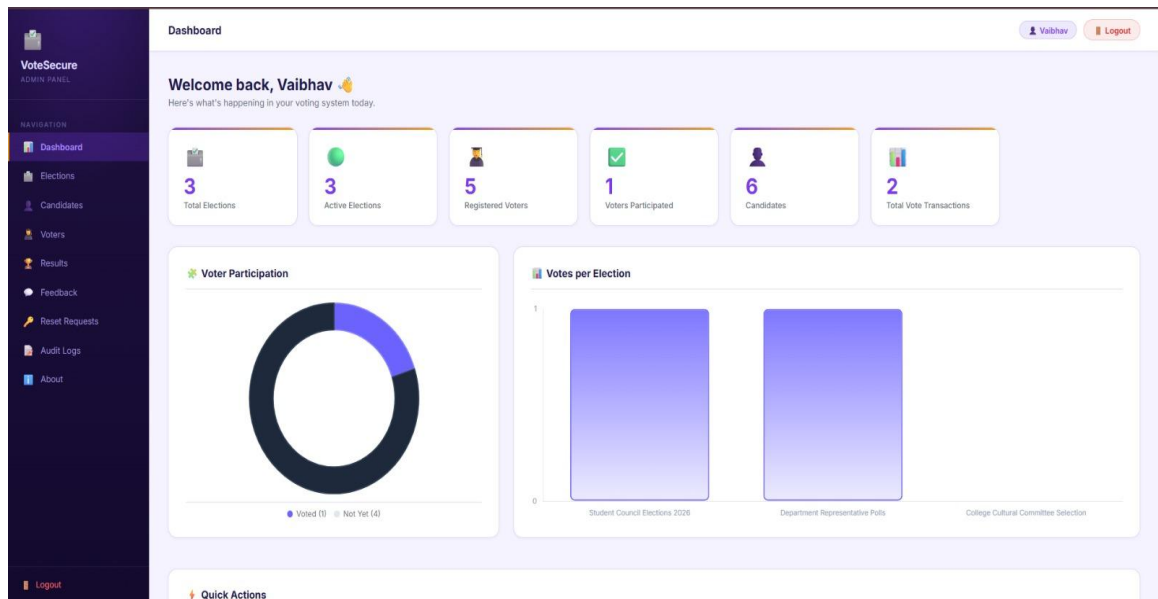


FIG 6.4 ADMIN DASHBOARD OF ONLINE VOTING SYSTEM

7. CONCLUSION & FUTURE SCOPE

7.1 CONCLUSION

Beyond core web application development, this project marks a significant technological advancement by implementing modern Cloud-Native and DevSecOps engineering practices. The entire application was containerized using Docker, orchestrating decoupled web and database microservices inside Amazon Elastic Kubernetes Service (EKS). A fully automated Jenkins CI/CD pipeline guarantees seamless continuous delivery, incorporating pre-flight syntax checks, automated container image building, Trivy security vulnerability audits, and zero-downtime rolling updates. Furthermore, real-world deployment challenges—including database persistence, network port conflicts, credential

separation via Kubernetes Secrets, and automated health monitoring—were rigorously resolved. Consequently, the project stands as a robust, production-ready, and scalable solution that bridges modern full-stack web development with enterprise-grade cloud computing and DevOps automation.

The Cloud-Based Online Voting System Using AWS successfully demonstrates the end-to-end development, containerization, and automated cloud deployment of a secure, transparent, and highly available digital election platform. By combining a modern web frontend (HTML5, CSS3, JavaScript) with an efficient backend (PHP 8.2) and relational database (MySQL 8.0), the application delivers seamless voter registration, secure authentication, intuitive ballot casting, strict One User – One Vote validation, and instantaneous real-time result tabulation.

The implemented system allows voters to securely register, log in, view candidates, and cast their votes online from any internet-enabled device. The admin module enables efficient management of elections, candidates, voters, and voting results through a centralized dashboard. The One User – One Vote mechanism ensures fair and transparent election processing by preventing duplicate voting and maintaining vote accuracy.

The integration of AWS cloud services improved the availability, accessibility, and reliability of the system. AWS EC2 provided stable cloud hosting, AWS S3 supported file and image storage, and AWS IAM enhanced cloud security through permission management and controlled access. The testing phase confirmed that all major modules function correctly and the system performs effectively under normal usage conditions.

In future, the system can be further enhanced by implementing OTP-based authentication, Aadhaar verification, blockchain-based voting, mobile applications, advanced encryption, and AI-based fraud detection mechanisms. Additional AWS services such as AWS RDS, Load Balancer, and Auto Scaling can also be integrated to support large-scale elections and improve overall system scalability and performance. These future enhancements can make the system more secure, scalable, and suitable for wider real-world election environments.

7.2 FUTURE SCOPE

The Cloud-Based Online Voting System Using AWS can be further enhanced by adding advanced features and modern cloud technologies to improve security, scalability, and user experience. Although the current system successfully performs online voting operations, several improvements can make it suitable for larger and more secure real-world elections.

One of the major future enhancements is the implementation of OTP-based authentication using email or mobile numbers. This will provide an additional security layer and improve user verification during login and voting. Aadhaar verification or College ID verification can also be integrated to ensure voter authenticity and prevent fake registrations.

The system can be improved further by integrating blockchain technology for secure and tamper-proof vote storage. Blockchain can increase transparency and make the voting process more trustworthy by preventing unauthorized modifications to voting records.

A dedicated mobile application for Android and iOS platforms can also be developed to improve accessibility and allow users to vote directly from smartphones. Future versions can include multilingual support so that users from different regions can easily use the system in their preferred language.

Advanced security mechanisms such as password encryption, HTTPS communication, CAPTCHA verification, biometric authentication, and AI-based fraud detection can also be implemented to improve overall system security and reliability.

REFERENCES

1. D. Chaum, “Secret-Ballot Receipts: True Voter-Verifiable Elections,” *IEEE Security & Privacy*, vol. 2, no. 1, pp. 38–47, 2004.
2. R. L. Rivest and W. D. Smith, “Three Voting Protocols: ThreeBallot, VAV, and Twin,” in *USENIX Workshop on Accurate Electronic Voting Technology*, 2007.
3. K. Sako and J. Kilian, “Receipt-Free Mix-Type Voting Scheme,” in *Advances in Cryptology — EUROCRYPT*, 1995.
4. A. Juels, D. Catalano, and M. Jakobsson, “Coercion-Resistant Electronic Elections,” in *Proceedings of the ACM Workshop on Privacy in Electronic Society*, 2005.
5. S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” 2008.
6. Amazon Web Services, “Overview of Amazon Web Services,” 2023.
7. Amazon Web Services, “AWS Security Best Practices,” 2023.
8. M. Armbrust et al., “A View of Cloud Computing,” *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010.
9. A. Singh and K. Chatterjee, “Cloud Security Issues and Challenges: A Survey,” *Journal of Network and Computer Applications*, vol. 79, pp. 88–115, 2017.
10. N. Kshetri and J. Voas, “Blockchain-Enabled E-Voting,” *IEEE Software*, vol. 35, no. 4, pp. 95–99, 2018.
11. S. Schneider et al., “End-to-End Verifiable Internet Voting Systems: A Survey,” *IEEE Communications Surveys & Tutorials*, vol. 19, no. 3, pp. 1772–1796, 2017.
12. P. Ryan, D. Bismark, J. Heather, S. Schneider, and Z. Xia, “Prêt à Voter: A Voter-Verifiable Voting System,” *IEEE Transactions on Information Forensics and Security*, 2009.

APPENDIX

Appendix contains additional supporting information related to the project that helps in

understanding the implementation and deployment of the system. It includes screenshots, database details, AWS configuration information, code snippets, and technical outputs that are not included in the main chapters of the report.

The appendix section provides supplementary material used during project development and testing. It serves as a reference for project implementation and deployment activities.

Appendix Contents

Database schema and table structure

AWS EC2 configuration screenshots

AWS IAM user and permission screenshots

AWS S3 bucket screenshots

Admin dashboard screenshots

Voter login and voting page screenshots

Result page screenshots

Important PHP code snippets

Apache and MySQL configuration details

GitHub repository screenshots

Testing outputs and validation results

The appendix helps in providing practical evidence of the project implementation and supports the overall documentation of the Cloud-Based Online Voting System Using AWS.

aws-voting-advanced/

|— .env # Database credentials (not committed)

|— .gitignore # Excludes .env, uploads/*, etc.

|— README.md # This documentation

|— index.php # Public landing page

|

|— config/

| |— database.php # MySQL connection (reads .env, fallback defaults)

|

```

├── database/
│   └── aws_voting.sql      # Full schema + seed data
│
├── uploads/               # Candidate profile images (gitignored)
│
├── assets/
│   └── css/
│       ├── admin.css      # Admin design system (dark sidebar theme)
│       └── voter.css      # Voter portal + Landing page styles (merged)
│
├── admin/                 # — ADMIN PORTAL —
│   ├── login.php         # Admin login
│   ├── logout.php        # Admin session destroy
│   ├── dashboard.php     # Analytics overview (Chart.js)
│   ├── manage_elections.php # Election CRUD + toggle status
│   ├── manage_candidates.php # Candidate CRUD + image upload
│   ├── manage_voters.php  # Voter list (with Aadhar/Mobile), filters, search
│   ├── add_voter.php      # Manual voter registration
│   ├── results.php        # Live vote counting + winner highlights
│   ├── export_results.php # Excel results download
│   ├── export.php         # Voter list Excel export
│   ├── feedback.php       # Voter feedback inbox
│   ├── reset_requests.php # Password reset request management
│   ├── logs.php           # Admin audit trail
│   ├── about.php          # System info & developer credits
│   └── includes/
│       ├── header.php     # Admin sidebar + topbar
│       └── footer.php     # Admin footer
│
└── voter/                 # — VOTER PORTAL —
    ├── login.php         # Voter login (Email + Password)
    ├── logout.php        # Voter session destroy
    ├── register.php       # Registration (Name/Email/Aadhar/Mobile/Password)
    ├── forgot_password.php # Password reset request form.
    │
    ├── dashboard.php      # Active elections list
    └── vote.php           # Voting ballot interface

```

- |— vote_success.php # Post-vote confirmation screen
- |— already_voted.php # Guard page for completed votes
- |— profile.php # Account details + password change
- |— feedback.php # Feedback submission
- |— includes/
 - |— header.php # Voter navbar + auth guard
 - |— footer.php # Voter footer